

Tut08

Aufgabe 9 – Protobuf

Benutzen Sie die IntelliJ Vorlage und erstellen Sie im Ordner `src/main/proto` eine Datei `users.proto`, mit der sie die Daten aus Aufgabe 11 speichern können. Schreiben Sie dann eine `main`-Methode, die die User in eine Datei schreibt (z. B. `user.bin`) und eine andere `main`-Methode, die diese Datei einliest und die User auf der Konsole ausgibt.

id	name	address
1	Müller	Englerstraße 11
2	Meyer	Daimlerweg 50
3	Kunz	Fährweg 12
4	Schmidt	Ettlinger Straße 90

```
package org.example;  
option java_outer_classname = "SchoolProto";
```

```
message Student { //generierte Java Klasse heißt Student  
    optional int32 id = 1;  
    optional string first_name = 2;  
    optional string last_name = 3;  
    optional string email = 4;  
}
```



```
Student s = Student.newBuilder()  
    .setId(1)  
    .setFirstName("Anna")  
    .setLastName("Müller")  
    .setEmail("anna@example.com")  
    .build();
```

```
message SchoolClass {  
    optional string name = 1;  
    repeated Student students = 2;  
}
```



```
SchoolClass sc = SchoolClass.newBuilder()  
    .setName("Informatik 1")  
    .addStudents(s) //fügen Student einzeln ein  
    .build();
```

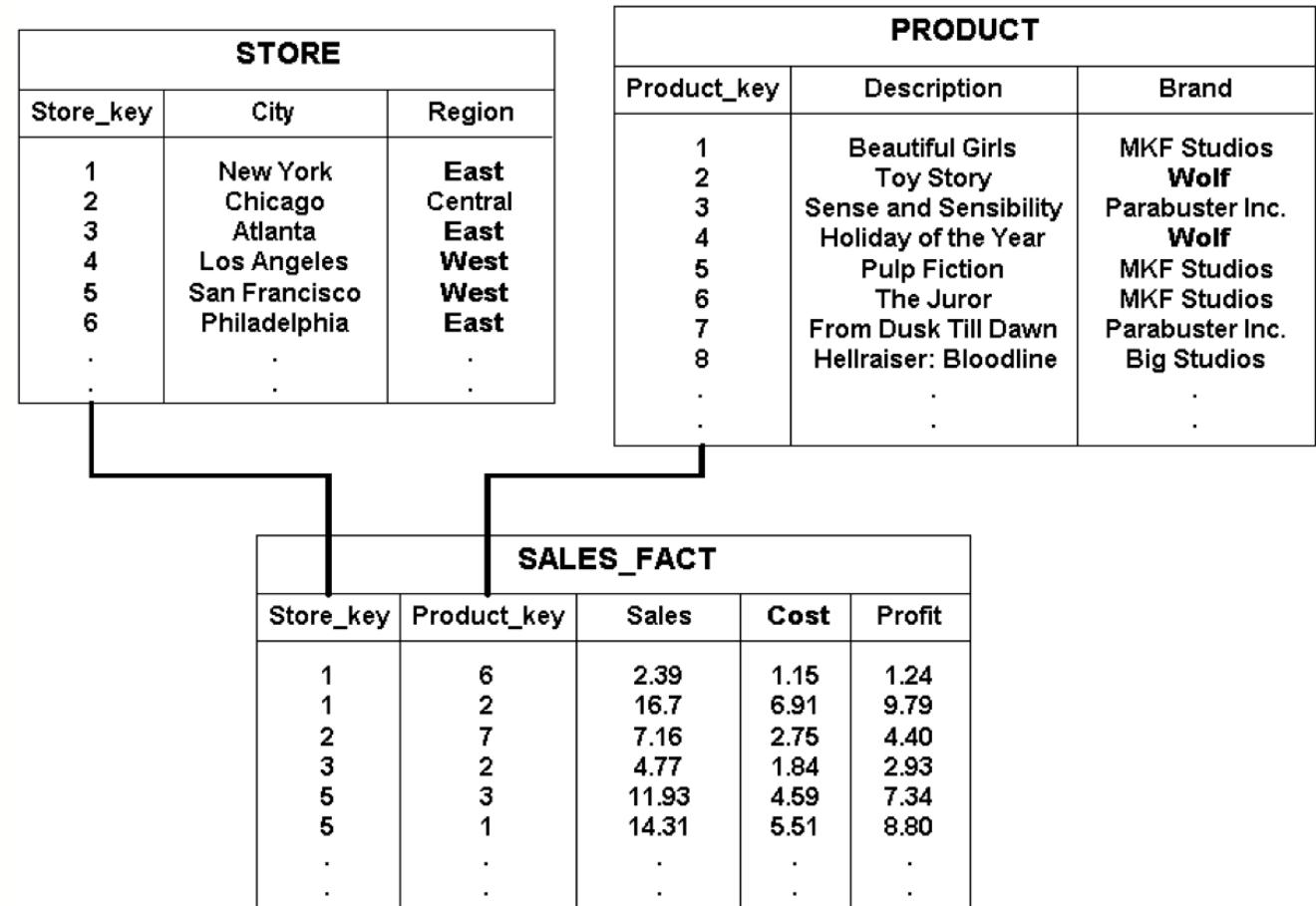
Datenbank

- Daten **speichern**
- Daten **wiederfinden**(auch bei Millionen von Einträgen schnell)Daten
- **schützen** (nicht jeder darf alles sehen)
- Daten **konsistent hält** (keine Widersprüche)

RDBMS

speichert Daten in Tabellen

- Komplexe Queries mit JOINS
- **Stärken:** beliebige Queries möglich, immer korrekt, gut erforscht
- **Schwächen:** skaliert schlecht bei riesigen Datenmengen



Datenbanken

Column stores (großer key Value store)

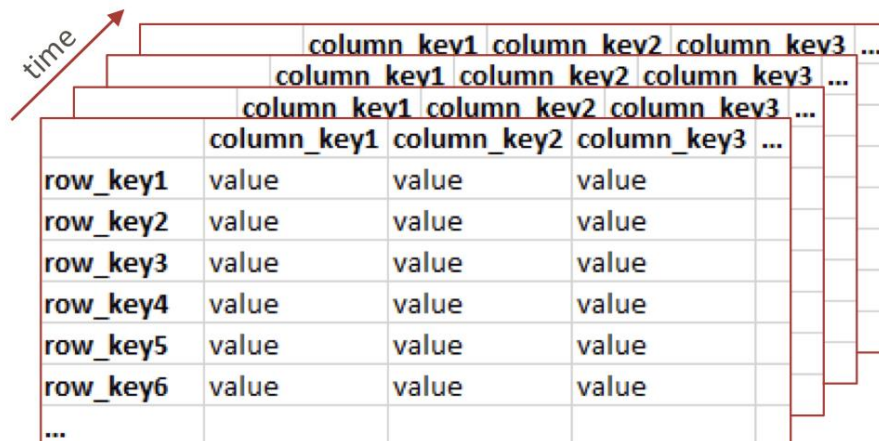
- Skaliert gut
- Einfache Queries möglich
- Kein JOIN

BSP: Key: "user_123"

→ name: "Anna"

→ city: "Berlin"

→ score: 42



Key-value store

- Schnell
- Skaliert gut
- Keine komplexen Queries (Abfragen) möglich

Key (Product ID)	Value (Quantity On-Stock)
9194-1985-1224	115
3495-5827-2525	1957
2957-5927-5472	0
3857-6285-5106	1959
...	...

Document Store

➔ Ganze JSON-Dokumente werden gespeichert

BSP:

```
{  
  "titel": "Inception",  
  "jahr": 2010,  
  "schauspieler": ["DiCaprio", "Hardy", "Page"],  
  "bewertung": 8.8  
}
```

```
{  
  "titel": "Kurzfilm XY",  
  "jahr": 2020  
}
```

- **Stärken:** flexibel, perfekt für Web-Entwicklung mit JSON
- **Schwächen:** ähnliche Skalierungsprobleme wie RDBMS
- **Wann benutzen:** Filmmetadaten, Produktkataloge, Web-Apps

Time Series DB

- Optimiert für Daten die **regelmäßig mit einem Timestamp** ankommen:

2026-06-03 12:00:00 → Temperatur 22°C

2026-06-03 12:00:10 → Temperatur 22.3°C

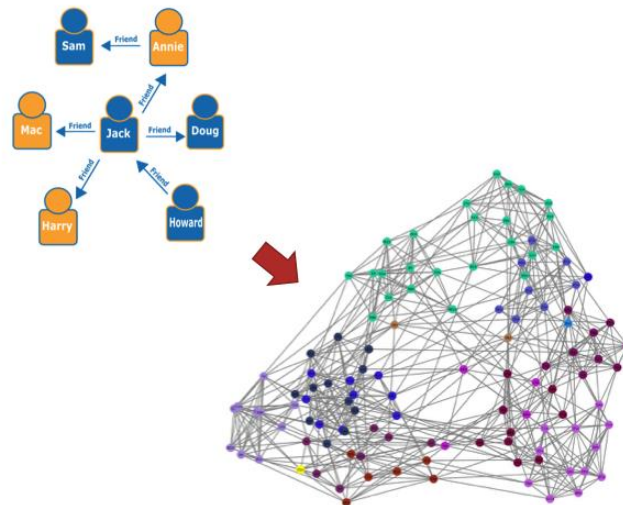
2026-06-03 12:00:20 → Temperatur 21.9°C

- IoT-Sensoren, Serverlogs, Börsenkurse → alles Zeitreihendaten
- **Stärken:** optimiert für zeitbasierte Queries („zeig mir die letzten 24h“)

Graph DB

- Optimiert für **Beziehungen zwischen Objekten**
- **Stärken:** perfekt für vernetzte Daten
- **Wann benutzen:** soziale Netzwerke, Betrugserkennung, Produktempfehlungen

Anna → kennt → Max
Max → kennt → Lisa
Lisa → kennt → Anna



Geographic DB

- Optimiert für **räumliche Daten**: wo ist was auf der Welt?
- Query: „zeig mir alle Cafés im Umkreis von 500m“
- **Wann benutzen:** Karten, Navigation, Fahrradrouten analysieren

"Café Müller" → 52.5200° N, 13.4050° O (Berlin)
"Café Schmidt" → 48.1351° N, 11.5820° O (München)

Zusammengefasst

- Korrektheit & komplexe Queries -> RDBMS
- Schneller Zugriff per Key -> Key-Value
- Große Datenmengen -> Column Store
- Flexible JSON-Dokumente -> Document Store
- Daten mit Timestamp -> Time Series
- Vernetzte Daten -> Graph DB
- Räumliche Daten -> Geographic DB

3 Datenbanken und Transaktionen

Aufgabe 10 – Datenbanken

Erklären Sie den Unterschied zwischen relationalen Datenbanken und Key-value-Datenbanken. Wofür sind sie jeweils besonders gut geeignet? Geben Sie außerdem an, welcher Datenbanktyp für folgende Use Cases am besten geeignet ist.

1. Für jede Userin einer Web-Anwendung den aktuellen Status der Anwendung speichern Key-Value
2. Filmmetadaten für einen Streaming-Service speichern Document Store
3. IoT-Sensoren produzieren regelmäßig Daten, die gespeichert werden sollen Time Series DB
4. Analyse von Fahrradfahrten zum Erkennen von unsicheren Stellen Geographic DB
5. Analyse von Transaktionen zur Erkennung von Betrug Graph DB
6. Eine Bank speichert die Konten und Transaktionen der Kund:innen RDBMS
7. Große Mengen an Daten aus Webanwendungen und Social Media sollen in Echtzeit analysiert werden, um Trends zu erkennen Column Store
8. Ein Krankenhaus speichert Daten über Patienten wie ihre vorherigen Diagnosen, Testergebnisse, usw. Die Daten müssen in Echtzeit von Mitarbeitenden aufgerufen werden RDBMS
9. Vor eine Webseite soll eine Caching-Datenbank geschaltet werden, in der die wichtigsten Daten für jeden User gespeichert sind, um schnellen Zugriff zu gewähren Key-Value
10. „Kundinnen, die dieses Produkt kauften, kauften auch folgendes Produkt:“ in einem Webshop Graph DB

SQL-Basics

Wenn alle
Spalten:
SELECT *

SELECT – **was** willst du sehen (welche Spalten)
FROM – **woher** (welche Tabelle)
WHERE – **welche Zeilen** (optional, filtert Ergebnisse)

- Grundstruktur:
- **SELECT** <spalten> **FROM** <tabellenname> **WHERE** <bedingung>;

Aufgabe 11 – SQL

Betrachten Sie die folgende Tabelle mit dem Namen Customers.

id	name	address
1	Müller	Englerstraße 11
2	Meyer	Daimlerweg 50
3	Kunz	Fährweg 12
4	Schmidt	Ettlinger Straße 90

SELECT * FROM Customers;
SELECT * FROM Customers WHERE id= 2;
SELECT address, name FROM Customers;

Schreiben Sie

1. eine SQL-Anfrage, die alle Einträge der Tabelle liefert,
2. eine SQL-Anfrage, die alle Einträge mit der id 2 liefert, und
3. eine SQL-Anfrage, die alle Namen und Adressen ausgibt. Die Anfrage soll zuerst die Adresse und dann den Namen liefern.

Object relational mapping

- Meistens:
 - jede Klasse wird zur Database
 - Jedes Feld zur Spalte
 - Jede Instanz des Objekts wird zur Zeile

Join Table:

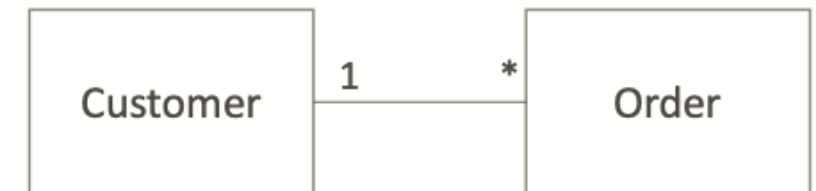
Customer		
ID	NAME	
1	John Doe	
2	Jane Doe	

Order		
ID	ADDRESS	CUSTOMER_ID
1	some address	1
2	some address	2
3	Some address	1

A red dashed arrow labeled "foreign key" points from the **CUSTOMER_ID** column in the Order table to the **ID** column in the Customer table.

```
public class Customer {  
    private int id;  
    private String name;  
    private Collection<Order> orders;  
  
    // getters and setters  
}
```

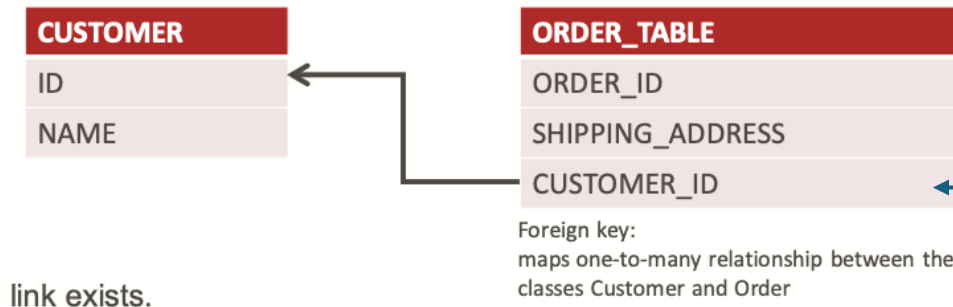
```
public class Order {  
    private int id;  
    private String address;  
    private Customer customer;  
  
    // getters and setters  
}
```



JPA

- standard, nicht implementation
- JPA -> mit Java Objekten arbeiten (entities) -> annotations
- jakarta.persistence.EntityManager

- **@Entity:** Klasse ist entity
- **@Id:** primary key
- **@OneToMany:** definiert Assoziation zu anderer Entity-Klasse
 - Z.B ein Customer hat viele Orders
- **@ManyToOne:** definiert Assoziation zu anderer Entity-Klasse
 - z.B. viele Orders gehören zu einem Customer
- **@Table(name = "Order_TABLE")** default: Klassenname als Tabellennamen
- **@Column(name = "ORDER_ID", unique = true)** default: Feldname als Spaltenname
- **@JoinColumn(name = "CUSTOMER_ID"):**
 - default: *Beziehungsname auf Eigentümerseite* + "_" + *Name der Primärschlüsselspalte der anderen Seite*



```

@Entity
public class Customer {
    @Id
    private int id;
    private String name;
    @OneToMany(mappedBy = "customer")
    private Collection<Order> orders;

    // + Getters and Setters
}

```

Sagt: schau bei
customer
Objekt

```

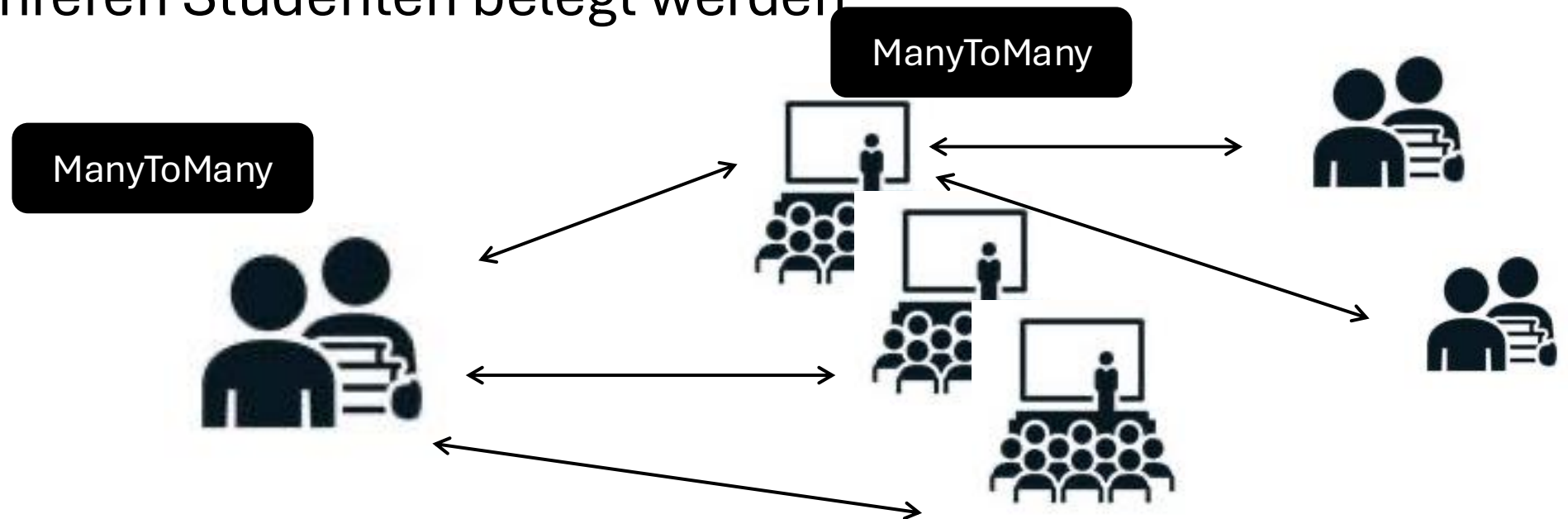
@Entity
@Table(name = "ORDER_TABLE")
public class Order {
    @Id
    @Column(name = "ORDER_ID")
    private int id;
    @Column(name = "SHIPPING_ADDRESS")
    private String address;
    @ManyToOne
    @JoinColumn(name = "CUSTOMER_ID")
    private Customer customer;
}

```

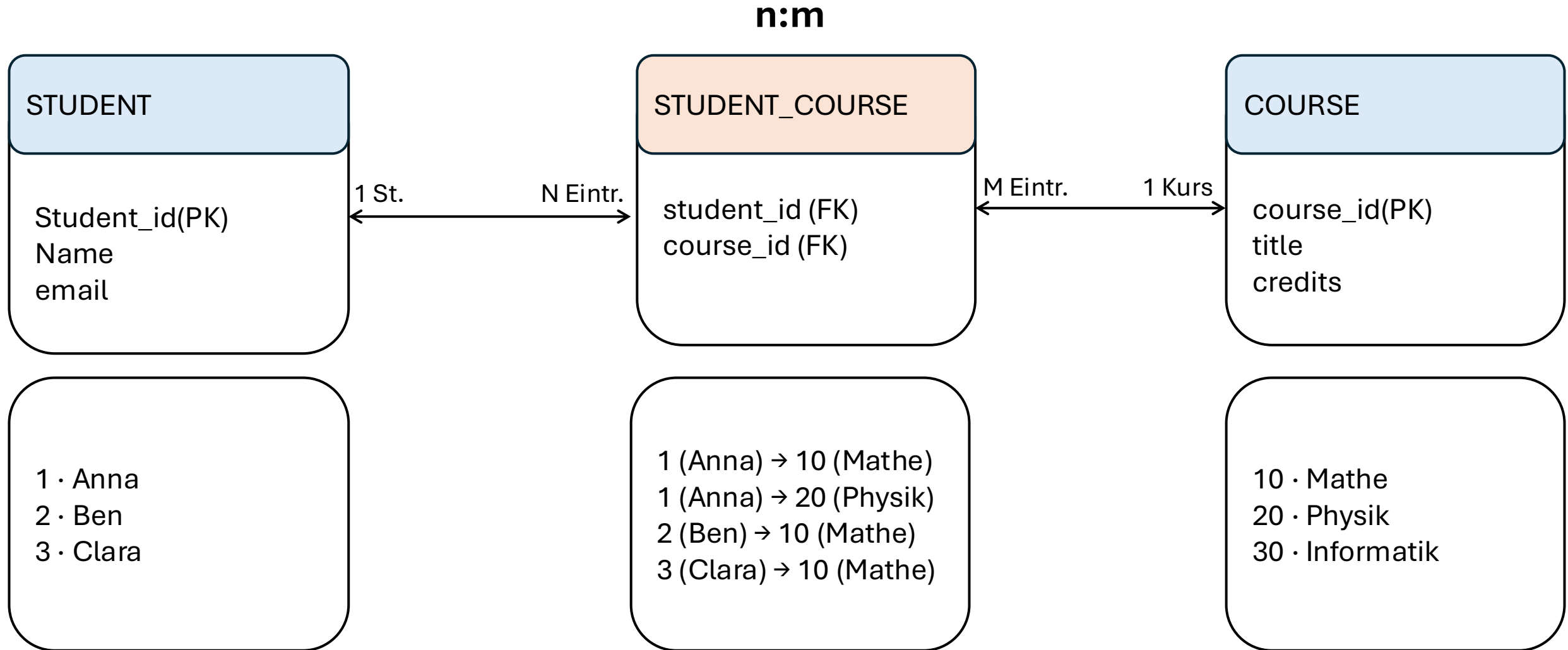
Verwaltet fremdschlüssel → Order
ist owner
(immer ManyToOne)

Zusatz

- ManyToMany: mehrere Datensätze der einen Tabelle mit mehreren Datensätzen der anderen Tabelle verknüpft
- **Beispiel:** Student kann mehrere Kurse belegen, und ein Kurs kann von mehreren Studenten belegt werden



ManyToMany wird aufgelöst durch Zwischentabelle




```

@Entity
public class Student {
    @Id
    private int id;

    @ManyToMany
    @JoinTable(
        name = "STUDENT_COURSE",
        joinColumns = @JoinColumn(name = "STUDENT_ID"),
        inverseJoinColumns = @JoinColumn(name = "COURSE_ID")
    )
    private List<Course> courses;
}

```

Student ist owner, es wird von Student aus zu courses navigiert

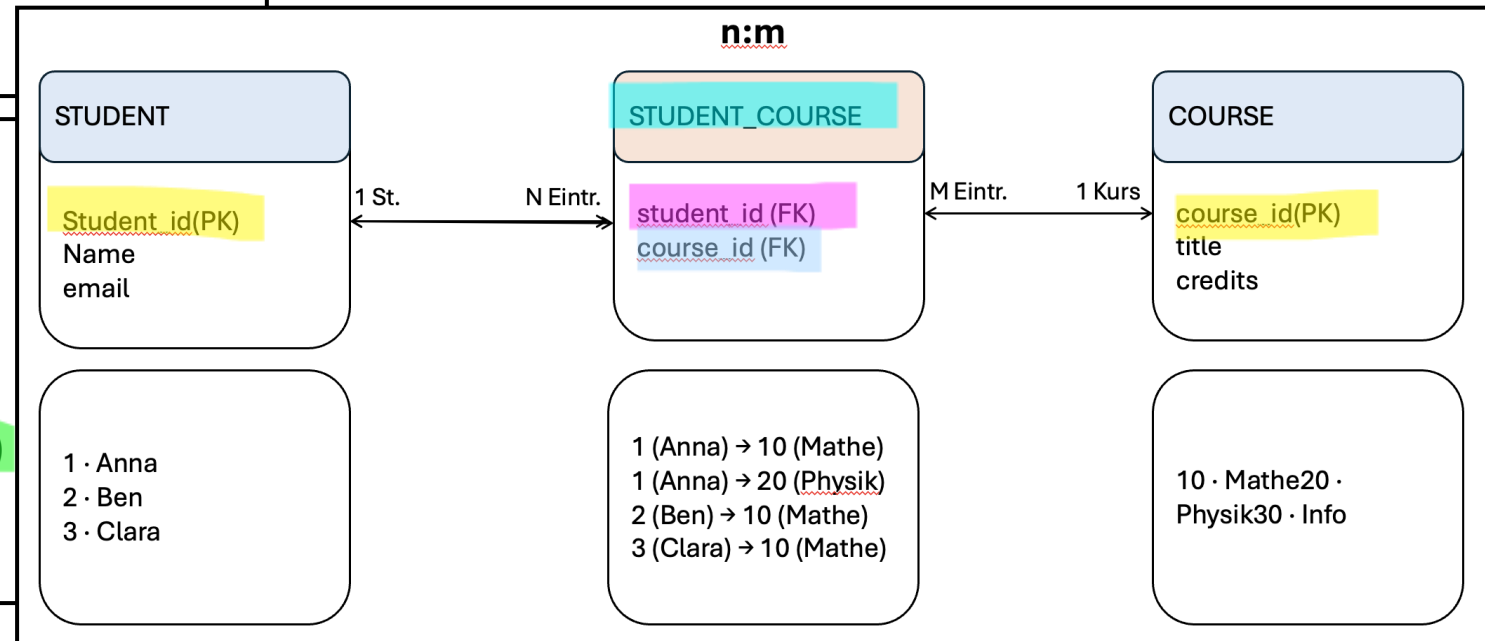
```

@Entity
public class Course {
    @Id
    private int id;

    @ManyToMany(mappedBy = "courses")
    private List<Student> students;
}

```

Kein eigens join Table nötig



Aufgabe 12 – ORM

Gegeben seien folgende Relationen einer Datenbank. Zusätzlich sind die folgenden Informationen bekannt.

- **StudentId** ist Primärschlüssel in der Tabelle Studentin.
- **VorlesungsId** ist Primärschlüssel in der Tabelle Vorlesung.
- Die Tabelle belegt verweist mit zwei Fremdschlüsseln auf die beiden anderen Tabellen.
- Jede:r Student:in kann mehrere Vorlesungen belegen und jede Veranstaltung kann von mehreren Studenten belegt werden (**Many-to-many-Beziehung**).

VorlesungsId	Name
1000	Programmieren 1
1001	Datenbanksysteme
1002	Anwendungssysteme
1003	Fog Computing
1004	Distributed Systems Prototyping
1005	Cloud Service Benchmarking

StudentId	Name
123456	Max Musterman
234567	Maja Musterfrau

VorlesungsId	StudentId
1000	123456
1001	123456
1002	123456
1005	123456
1004	234567
1006	234567
1003	234567
1000	234567

Wir wollen dass
Vorlesung owner ist

Auf die bestehende Datenbank soll nun zugegriffen werden. Hierzu ist es nötig, die vorliegenden Daten mithilfe von **Object-Relational Mapping** auf Objekte abzubilden. Ergänzen Sie die zwei Klassen Vorlesung und Student so, dass die Information, welcher Student welche Vorlesung besucht, nicht verloren geht (die andere Richtung darf vernachlässigt werden). Fügen Sie entsprechende **Annotations** ein, so dass die Klasse als **Persistence Entity** korrekt deployed werden kann. Die Spezifikation für Java Beans soll erfüllt werden.

```
@Entity
@Table(name="Student")
```

```
public class Student implements Serializable {
    private int matrikelnummer = 0;
    private List<Vorlesung> vorlesungen = null;
    private String name = null;
```

```
    public Student () {
        vorlesungen = new ArrayList<>();
    }
```

```
@ManyToMany(mappedBy="students")
```

```
    public List<Vorlesung> getVorlesungen() { return this.vorlesungen; }
```

```
    public void setVorlesungen(List<Vorlesung> vorlesungen) { this.vorlesungen = vorlesungen; }
```

```
@Column(name="Name")
```

```
    public String getName(){ return this.name; }
```

```
    public void setName(String name) { this.name = name; }
```

```
@Id
@Column(name="MatrNr",unique =true)
```

```
    public int getMatrikelnummer() { return this.matrikelnummer; }
```

```
    public void setMatrikelnummer (int matrikelnummer) { this.matrikelnummer = matrikelnummer; }
```

```
}
```

- **@Entity:** Klasse ist entity
- **@Id:** primary key
- **@OneToMany:** definiert Assoziation zu anderer Entity-Klasse
 - Z.B. ein Customer hat viele Orders
- **@ManyToOne:** definiert Assoziation zu anderer Entity-Klasse
 - z.B. viele Orders gehören zu einem Customer

- **@Table(name = "Order TABLE")** default: Klassenname als Tabellename
- **@Column(name = "ORDER_ID")** default: Feldname als Spaltenname
- **@JoinColumn(name = "CUSTOMER_ID"):**
 - default: Beziehungsname auf Eigentümerseite + "_" + Name der Primärschlüsselspalte der anderen Seite

```
@Entity
@Table(name = "Vorlesung")
```

```
public class Vorlesung implements Serializable {
```

```
    private int vorlesungsnummer = 0;
    private String name = null;
    private List<Student> students = ...;
```

```
@Column(name = "Name")
```

```
    public String getName(){return this.name;}
```

```
    public void setName(String name) {this.name = name;}
```

```
@Id
@Column(name = "VorlNr", unique = true)
```

```
    public int getVorlesungsnummer() {return this.vorlesungsnummer;}
```

```
@ManyToMany
```

```
    public List<Student> getStudents () {return this.students;}}
```

Verwaltet die join Tabelle -> es wird von
Vorlesung zu student navigiert

- **@Entity:** Klasse ist entity
- **@Id:** primary key
- **@OneToMany:** definiert Assoziation zu anderer Entity-Klasse
 - Z.B. ein Customer hat viele Orders
- **@ManyToOne:** definiert Assoziation zu anderer Entity-Klasse
 - z.B. viele Orders gehören zu einem Customer

- **@Table(name = "Order TABLE")** default: Klassenname als Tabellename
- **@Column(name = "ORDER_ID")** default: Feldname als Spaltenname
- **@JoinColumn(name = "CUSTOMER_ID"):**
 - default: Beziehungsname auf Eigentümerseite + "_" + Name der Primärschlüsselspalte der anderen Seite

Aufgabe 13 – ACID

Beschreiben Sie kurz die vier ACID-Eigenschaften (Atomicity, Consistency, Isolation, Durability). Überlegen Sie auch, warum diese für Enterprise-Anwendungen jeweils wichtig sind.

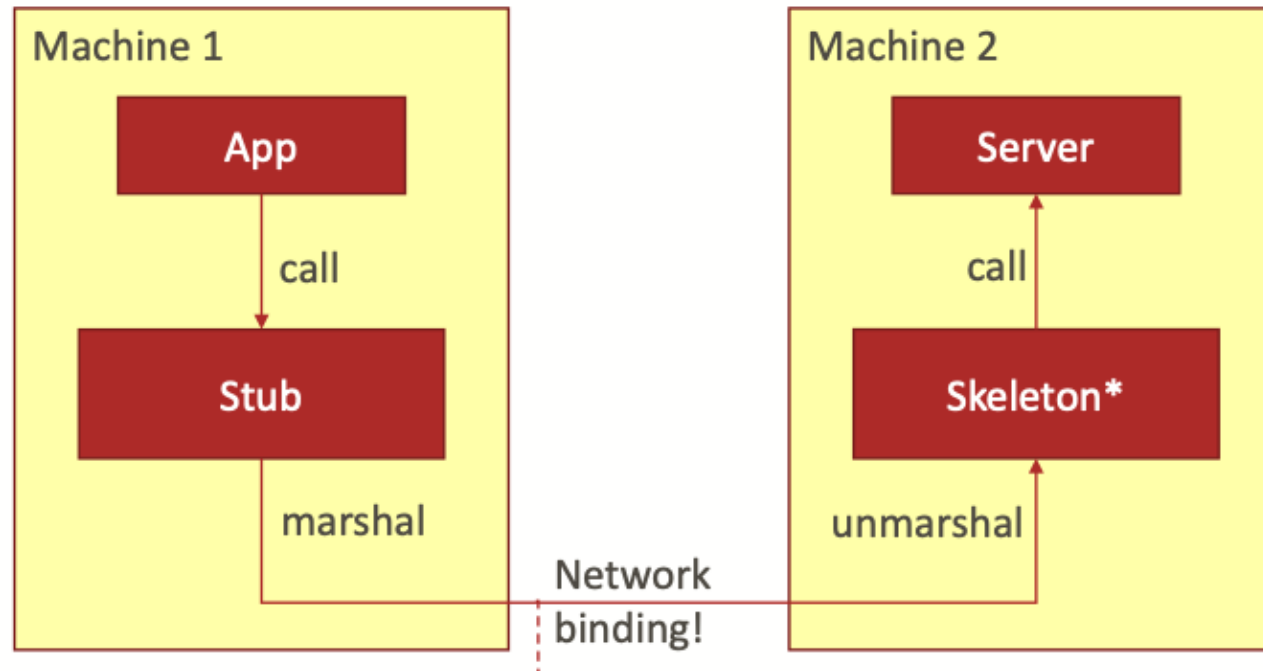
- **Atomicity:** Jede Transaktion läuft ganz oder gar nicht ab, sie ist also unteilbar.
 - **Consistency:** Eine Datenbank ist vor und nach der Transaktion in einem konsistenten Zustand, in dem alle Integritätsbedingungen gewahrt werden.
 - **Isolation:** Transaktionen laufen isoliert voneinander ab, d.h. keine Transaktion kann auf unfertige Zwischenergebnisse einer anderen Transaktion zugreifen.
 - **Durability:** Alle Änderungen einer Transaktion sind dauerhaft gespeichert, sobald die Transaktion abgeschlossen ist.
- Diese Eigenschaften gewährleisten die Zuverlässigkeit und Stabilität von Datenbankoperationen, was für Enterprise-Anwendungen extrem wichtig ist.

RPC: Remote Procedure Call

- Zwei Prozesse wollen auf verschiedenen Maschinen kommunizieren
 - Mit Sockets → viel Arbeit
- Ziel: rufe Funktion auf Maschine B auf, als wäre sie lokal
 - brauchen Abstraktion
 - Communication Middleware
 - übernimmt Serialisierung, Transport, Fehlerbehandlung, und gibt nach außen normale Funktion zum Aufrufen

Aus IDL generiert Compiler Stubs und Skeletons für jede gewünschte Sprache
-> Python Client ruft Java Server auf -> **kein problem**

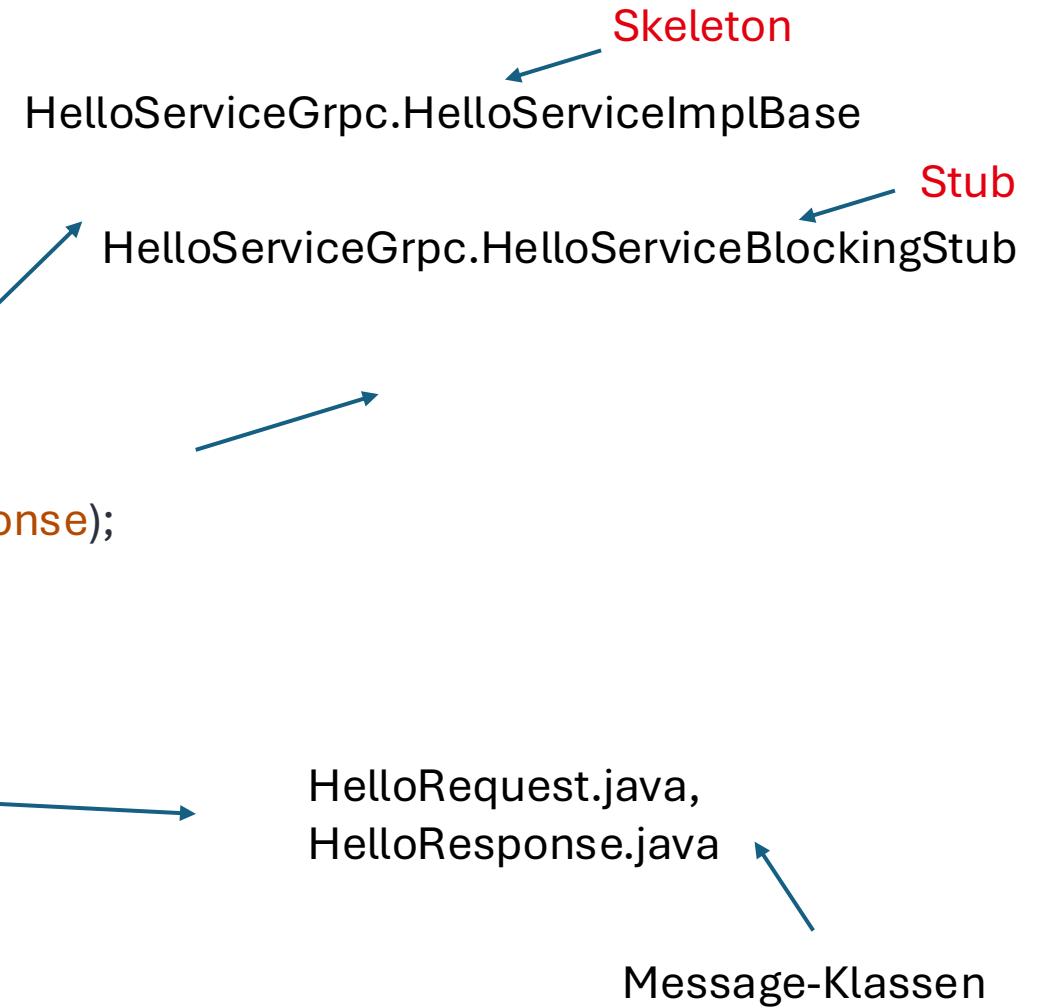
RPC and middleware



gRPC

- IDL: Protocol buffers (proto3)

```
service HelloService {  
    rpc SayHello (HelloRequest) returns (HelloResponse);  
}  
  
message HelloRequest {  
    string greeting = 1;  
}  
  
message HelloResponse {  
    string reply = 1;  
}
```



Was wir schreiben müssen:

- Auf **Serverseite** generierten Skeleton extenden:

```
public class MyServer extends HelloServiceGrpc.HelloServiceImplBase {  
  
    @Override  
    public void sayHello(HelloRequest req, StreamObserver<HelloResponse> resp) {  
        // hier Logik  
    }  
}
```

- Grpc unterstützt Streaming -> Streamobserver -> Objekt auf das wir Antworten drauf schreiben
- **resp**.onNext(response): schick eine Antwort
- **resp**.onCompleted(): Ich bin Fertig
- **resp**.onError(e): Fehler aufgetreten

Für Zuhause und die Hausaufgabe

[SoSe 2026] AS / Übungsbetrieb / Übungsaufgabe 2

Übungsaufgabe 2

Aufgabe

Einstellungen

Einreichungen

Erweiterte Bewertung

Mehr ▾

Geöffnet: Mittwoch, 3. Juni 2026, 00:00

Fällig: Freitag, 3. Juli 2026, 18:00



hello-world.zip

8. Juni 2026, 12:51



ue2-communication.pdf

3. Juni 2026, 11:53

Bewerten

Wir schreiben: `hello.proto`



protoc-Compiler: `HelloServiceGrpc.java` ← Stub + Skeleton drin
 `HelloRequest.java`
 `HelloResponse.java`



Wir schreiben: `MyServer` extends `HelloServiceImplBase` (Serverlogik)
 `MyClient` benutzt `HelloServiceBlockingStub`



Laufzeit:

Client-Code ruft `stub.sayHello(req)` auf



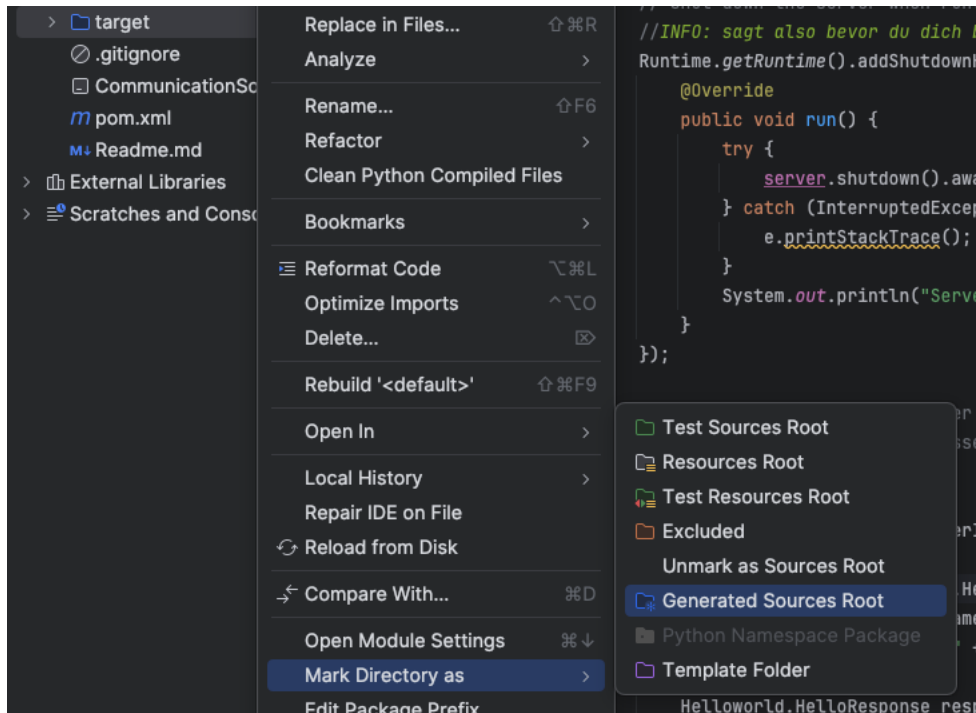
(Stub serialisiert, schickt über HTTP/2)

Skeleton empfängt, deserialisiert

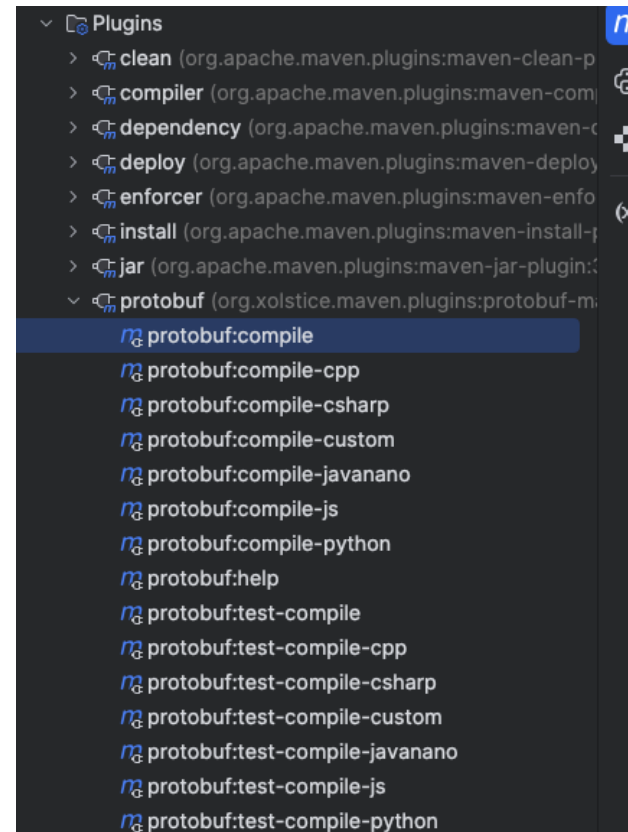


`MyServer.sayHello()` wird aufgerufen

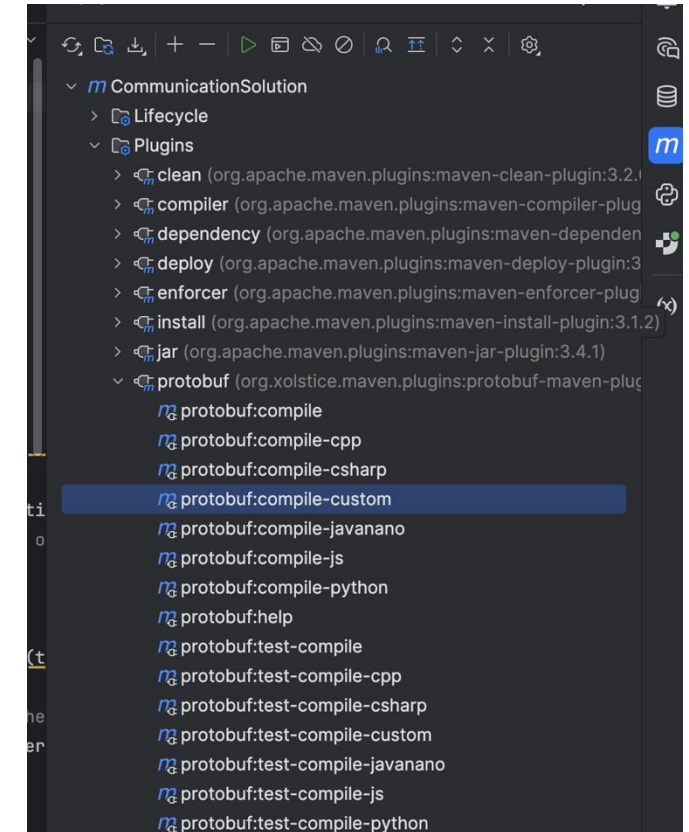
Deine Klassen finden deine .proto nicht ?



Target schon als
generated Sources Root
markiert ?



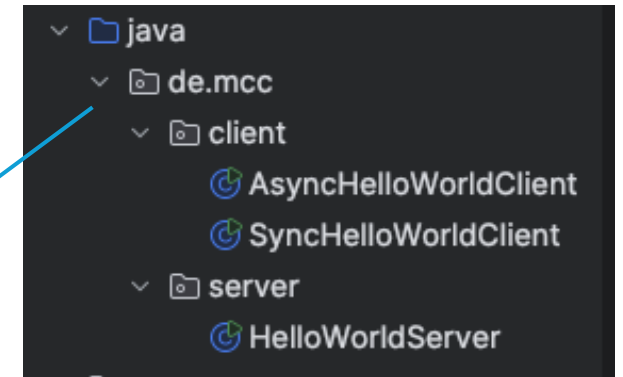
Bei m (rechts) ->
communicationSolution -> Plugins -
> protobuf -> protobuf:compile
gedrückt?



Bei m (rechts) ->
communicationSolution -> Plugins -
> protobuf -> protobuf:compile -
custom gedrückt?

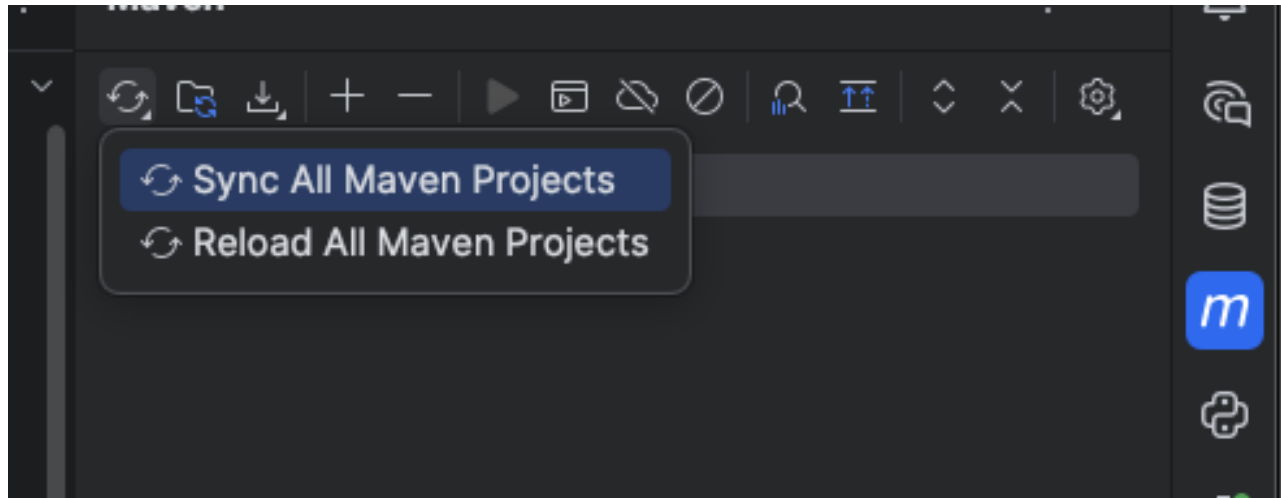
Und vergesse diese Zeilen nicht:

```
syntax = "proto3";  
package de.mcc;  
option java_package = "de.mcc";
```



Java_package = ____;
ist der Pfad zu deinen Klassen

Irgendetwas in pom.xml ist rot?



GitHUb –
Code kommt
später

